

Lightweight Secure Aggregation for Personalized Federated Learning with Backdoor Resistance

Tingyu Fan^{*†‡}, Xiaojun Chen^{*†‡¶}, Ye Dong[§], Xudong Chen^{*†}, Yuexin Xuan^{*†‡}, and Weizhan Jing^{*†‡}

^{*}Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

[†]Key Laboratory of Cyberspace Security Defense, Beijing, China

[‡]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

[§]iTrust, Singapore University of Technology and Design, Singapore

{fantingyu, chenxiaojun, chenxudong, xuanyuexin, jingweizhan}@iie.ac.cn, ye_dong@sutd.edu.sg

Abstract—Existing federated learning (FL) systems are highly vulnerable in terms of security and privacy due to their distributed architecture, facing poisoning attacks and inference attacks from adversaries. Some prior works have combined poisoning defenses with cryptographic tools: Secure Multi-Party Computation, Zero-Knowledge Proof, and Homomorphic Encryption to propose robust secure aggregation methods that provide security and privacy preservation for FL. Recently, Qin *et al.* (KDD’23) demonstrate that personalized federated learning (pFL) can effectively resist backdoor injection in poisoning attacks. In this paper, we analyze that as the number of malicious attackers increases, pFL remains vulnerable to backdoor attacks. Moreover, we reveal that current robust secure aggregation methods fail to offer efficient and robust backdoor defense for pFL. Therefore, we propose FLIGHT, a robust secure aggregation method for pFL. It implements a lightweight backdoor detection through a two-stage personalized defense mechanism and ensures privacy preservation using communication-efficient two-party secure computation (2PC) protocols. Extensive experiments on diverse datasets and neural networks validate that FLIGHT decreases run-time up to $64\times$ compared by prior work RoFL (S&P’23), and $42\times$ compared to FLAME (USENIX Security’22).

Index Terms—Secure Aggregation, Personalized Federated Learning, Backdoor Attack, Security and Privacy

1. Introduction

Federated Learning (FL) is an emerging distributed architecture used to collaboratively train machine learning (ML) models across multiple clients while preserving their data privacy. Each client trains a local model and uploads its model update to a central server, where the server aggregates these updates to obtain a global model. This approach avoids direct collection of client’s sensitive data, as the global model is then returned to the clients for further training rounds. This distributed nature of FL has found widespread applications in various privacy-sensitive scenarios, including

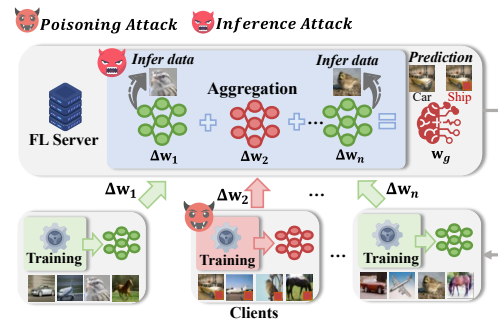


Figure 1: Two distinct types of attacks in federated learning.

medical image analysis [48], autonomous vehicles assistance [44], and anomaly detection in Internet of Things [40].

However, due to its distributed architecture, FL faces significant challenges in security and privacy. For example, in the field of image classification (*cf.* Figure 1), covering medical analysis and industrial detection, distributed clients use their private local images to train local models and then upload the model updates to the server. The server aggregates these updates to train a more generalized global model, which can better perform image classification tasks. However, during training, malicious clients compromised by adversary can launch poisoning attacks to tamper the global model: untargeted attacks [17] aim to reduce the main task accuracy (MA) on clean samples, while stealthier targeted (backdoor) attacks [23], [4], [53] seek to increase the backdoor task accuracy (BA) on poisoned samples without affecting the MA on clean samples, causing the global model to misclassify poisoned samples with specific triggers. Such attacks severely threaten the security of the global model. In terms of privacy, a compromised server can launch inference attacks [10], [20], attempting to infer the original private images from the model updates uploaded by clients. This clearly undermines the fundamental privacy-preserving nature of FL.

To defend against poisoning attacks, common approaches implement robust aggregation to filter out poisoned updates. FLTrust [9] assumes that the server possesses benign validation dataset, further removes updates

[¶]Xiaojun Chen is the corresponding author.

with large deviations from the direction of benign samples in linear complexity by computing the cosine similarity with each model update. Sun *et al.* utilizes the characteristic that the norm of poisoned updates differs from that of benign updates, excluding poisoned updates using predefined norm bound [49]. FLAME [41] strengthens resistance to poisoning attacks by using HDBSCAN clustering [7] to dynamically eliminate updates that deviate significantly from benign ones, and then applying norm clipping to further mitigate the impact of any remaining poisoned updates. Correspondingly, in the face of inference attacks, encrypting model updates using cryptographic tools can effectively prevent malicious server from inferring privacy data. Furthermore, some schemes combine robust aggregation with cryptographic tools Secure Multi-Party Computation (MPC) [21], Zero-Knowledge Proof (ZKP) [18], and Homomorphic Encryption (HE) [16] to achieve robust secure aggregation [36], preserving data privacy while providing robustness against poisoning attacks. For example, SecureFL [25] integrates MPC with FLTrust defense strategy, deploying cosine similarity detection method based on Secure Two-party Computation (2PC) [13], [39] protocols between client and server. RoFL encrypts model updates using HE and combines defense rule [49], verifying whether the norm of updates exceeds predefined bound through ZKP. FLAME employs two MPC servers and designs 2PC protocols to implement ciphertext clustering detection.

Moreover, in real-world scenarios, the private data of clients usually exhibits heterogeneous characteristics and presents Non-IID (Non-Independently and Identically Distributed) distribution, making it difficult for traditional FL to generate a generalizable global model with satisfactory performance. To address the issue of client data heterogeneity, personalized federated learning (pFL) presents a promising solution for customizing the personalized models for each client through Fine-tuning [34], Regularization [32], [50], and Parameter-decoupling [33], [43] of the global model. As a specialized form of FL, it is still necessary to analyze the security and privacy issues associated with pFL. In terms of security, recent work conducted by Qin *et al.* has demonstrated that some pFL methods can provide robustness against backdoor attacks [45]. However, these observations are only tested under poisoned model rate $\delta < 0.02$ (δ is the fraction of malicious clients m to the total clients n), which deviates from real-world attack scenarios. In order to further validate the robustness of pFL approaches, we raise the following questions:

Q1: How does the robustness of pFL against backdoor change with the increasing number of malicious clients? In Section 3, to simulate realistic scenarios of malicious client attacks, we evaluate various pFL schemes on CIFAR-10 against *Badnet Backdoor Attack* [23] across a wide range of poisoned model rate $\delta \in \{0.1, 0.2, 0.3, 0.4\}$. The experiments illustrate that as the number of malicious clients grows, existing pFL schemes fail to effectively resist backdoor attacks. The backdoor task converges after only a few training rounds, achieving a high accuracy of 80-99%.

Similarly, since pFL is also based on the paradigm of

server-aggregated parameters, it still faces privacy issues, namely the inability to resist inference attacks from malicious server. Therefore, we need to implement privacy-preserving pFL based on cryptographic secure aggregation, while also filtering out backdoor attack from malicious clients. Existing robust secure aggregation methods mentioned earlier, such as FLAME, RoFL, and SecureFL, provide privacy preservation and robustness against poisoning attacks for FL under IID data distributions. However, can these methods provide sufficient robustness guarantees for pFL under Non-IID data distributions? Additionally, considering these schemes rely on complex detection algorithms and cryptographic tools, can they provide high aggregation efficiency (low communication and run-time)? Thus, we pose the following question:

Q2: How do existing robust secure aggregation schemes applied to pFL perform regarding robustness and efficiency? In Section 3, we evaluate the robustness and efficiency of pFL when integrated with existing robust secure aggregation methods. The experiments show that while FLAME enhances backdoor defense through dynamic detection, it incurs significant communication and run-time overhead due to its complex clustering algorithm. Conversely, RoFL and SecureFL, undermine MA and backdoor defense in Non-IID setting, respectively, because of their small norm bound and fixed validation dataset. Notably, SecureFL is more efficient in secure aggregation, leveraging a linear detection algorithm.

Goals and Contributions. From the above observations, we find that: 1) The backdoor resistance of pFL decreases as the number of malicious clients increases; 2) Existing robust secure aggregation schemes struggle to balance robustness and efficiency for pFL. To address this challenge, we propose FLIGHT, aiming to combine the strengths of FLAME and SecureFL to achieve strong backdoor defense with lightweight complexity. Firstly, we design a two-stage personalized defense mechanism to enable efficient backdoor attack detection. In Stage1, a limited amount of clustering is performed to determine the direction of benign updates. In Stage2, historical updates are leveraged to dynamically construct next-round validation models for each client, determining whether the model update is from a malicious client by calculating the inconsistency between the actual update and its validation model with linear complexity. Furthermore, we design a series of communication-efficient 2PC protocols for this two-stage defense algorithm and deploy them on two MPC servers. By leveraging the two-stage feature, we only need a small amount of communication-intensive clustering in Stage1, allowing for a large number of communication-efficient linear detection protocols in Stage2. This reduces the overall communication cost for end-to-end training, achieving efficient and robust secure aggregation. In summary, we make the following contributions:

(1) We first observe that as the number of malicious clients increases, the robustness of pFL against backdoor attacks decreases. Additionally, we find that existing robust secure aggregation scheme fail to achieve an optimal trade-

off between backdoor resistance and aggregation efficiency for pFL.

(2) We propose FLIGHT, a two-stage personalized backdoor defense mechanism. By running a few rounds of clustering algorithms in Stage1, we enable a lightweight detection method with linear complexity in Stage2, which optimizes MPC communication.

(3) We design a series of 2PC detection protocols with linear complexity for FLIGHT: Π_{Cos} , Π_{NT} , and Π_{Softmax} to achieve privacy preservation. Leveraging the two-stage feature, we employ a small number of communication-intensive clustering protocol to enable a large number of communication-efficient linear detection protocols. This reduces the end-to-end training communication cost by $3\text{-}67\times$ compared to FLAME and RoFL.

(4) On the MNIST, Fashion-MNIST, and CIFAR-10 training tasks with Non-IID distribution, we demonstrate that FLIGHT¹ effectively defends against various backdoor attacks for pFL while reducing run-time by $2\text{-}64\times$ compared to FLAME, RoFL, and SecureFL.

2. Preliminaries

2.1. Personalized Federated Learning

Suppose FL has n clients, each client $C_i, i \in [1, n]$ has its own dataset $\mathcal{D}_i$. Initially, server broadcasts the global parameter $\mathbf{w}_g$ to each client C_i , where C_i locally train their parameters based on the $\mathbf{w}_g$ and private dataset $\mathcal{D}_i$:

$$\mathcal{L}_i(\mathbf{w}_g) = \mathbb{E}_{(x,y) \sim \mathcal{D}_i} [f_i(\mathbf{w}_g; x, y)] \quad (1)$$

Here, $\mathcal{L}_i(\mathbf{w}_g)$ represents the expected loss for each C_i , typically the cross-entropy loss for classification tasks. Then, C_i gets local model $\mathbf{w}_i$, while the purpose of server is to weight aggregate $\mathcal{L}_i(\mathbf{w}_g)$ to minimize the global objective function:

$$\min_{\mathbf{w}_g} \mathcal{L}_g(\mathbf{w}_g) = \sum_{i=1}^N \gamma_i \mathcal{L}_i(\mathbf{w}_g) \quad (2)$$

Various aggregation methods are employed to update global $\mathbf{w}_g$, for instance, FedAvg [37] sets the weight factor γ_i in Eq. 2 as $\frac{|\mathcal{D}_i|}{|\mathcal{D}|}$.

However, in Non-IID scenario (class skew), different participants have distinct tasks, making it challenging for standard FL to provide personalized models for each client. If clients C_i only obtains own model $\mathbf{w}_i$ through local training without sharing the global parameters $\mathbf{w}_g$ through communication, the limited number of local training samples can lead to models that are highly personalized but lack generalization. To strike the balance between generalization and personalization, personalized federated learning (pFL) customizes local models $\mathbf{w}_i$ for each participant based on the global model $\mathbf{w}_g$, thus better adapting to the Non-IID scenario. Depending on different personalized strategies, pFL can be categorized in such Fine-tuning, Regularization, Parameter-decoupling approaches:

1. Our code is available at <https://github.com/SimonB6/OpenFLIGHT>.

(1) *Fine-tuning* [34]. Clients perform standard FL to obtain a global model $\mathbf{w}_g$, which is then fine-tuned based on their respective private data to obtain personalized models. This approach preserves the generalization of the global model while making it more suitable for local tasks.

(2) *Regularization* [32], [50]. Initially, each client generates personalized models $\mathbf{w}_i$ using their local data $\mathcal{D}_i$, while global model $\mathbf{w}_g$ is also trained among all clients. To enhance the model's personalization, a regularization loss $\mathcal{R}$ is added to the local loss function, which describes the similarity between $\mathbf{w}_g$ and $\mathbf{w}_i$ to constrain the influence of local updates. For instance, Ditto [32] introduces the regularizer $\lambda \|\mathbf{w}_i - \mathbf{w}_g\|_2$ to improve the effectiveness of personalized models.

(3) *Parameter-decoupling* [33], [43]. This approach divides the client's local model $\mathbf{w}_i$ into two parts: shared parameters $\mathbf{w}_i^s$ and private parameters $\mathbf{w}_i^p$. The shared parameters $\mathbf{w}_i^s$ are shared among all clients in the FL system, while the private parameters $\mathbf{w}_i^p$ are trained locally on the clients. For example, FedBN [33] stores the BN layer parameters $\mathbf{w}_i^p$ locally while sharing the remaining parameters $\mathbf{w}_i^s$.

2.2. Adversary Attacks and Capabilities

Backdoor Attack. In this work, we focus on backdoor attack of poisoning attacks in FL, the malicious client compromised by $\mathcal{A}^c$ adds a backdoor trigger T on the selected subset of training samples, converting the target label y to an attacker-chosen label y' . This creates poisoned samples $(T(x), y')$ that cause the model to learn the main task while also learning the backdoor subtask, ultimately leading the poisoned global model $\mathbf{w}_g$ to predict the attacker-chosen label y' for the attacker-chosen test inputs $T(x)$.

$$\begin{aligned} \max_{\mathbf{w}_g} \text{MA} &= \mathbb{E}_{(x,y) \sim \mathcal{D}} [\Pr(f(\mathbf{w}_g; x)) = y] \\ \max_{\mathbf{w}_g} \text{BA} &= \mathbb{E}_{(x,y) \sim \mathcal{D}} [\Pr(f(\mathbf{w}_g; T(x)) = y')] \end{aligned} \quad (3)$$

The objective of this attack is to maximize the Backdoor Task Accuracy (BA) on poisoned samples, while maintaining the high Main Task Accuracy (MA) on benign samples. There are three common types of backdoor attacks on FL:

(1) *Badnet* [23]. This method injects a global trigger to the training samples and flips the label to the desired target class.

(2) *Scaling Attack* [4]. To enhance the impact of the backdoor attack on the global model, scaling attack scales up the weight of poisoned model updates by a factor of λ .

(3) *DBA* [53]. DBA decomposes the global trigger pattern into separate local patterns. Malicious clients implant these patterns into different dataset, targeting different neurons.

Inference Attack. Although FL systems protect private data through local training, the malicious sever compromised by $\mathcal{A}^s$ can exploit match terms with Euclidean and angular metrics to construct a loss function between the reconstructed data x^* and the actual data x derived from the model update. By continuously minimizing this loss function, the adversary is able to reconstruct the local data $\mathcal{D}_i$ [10], [20].

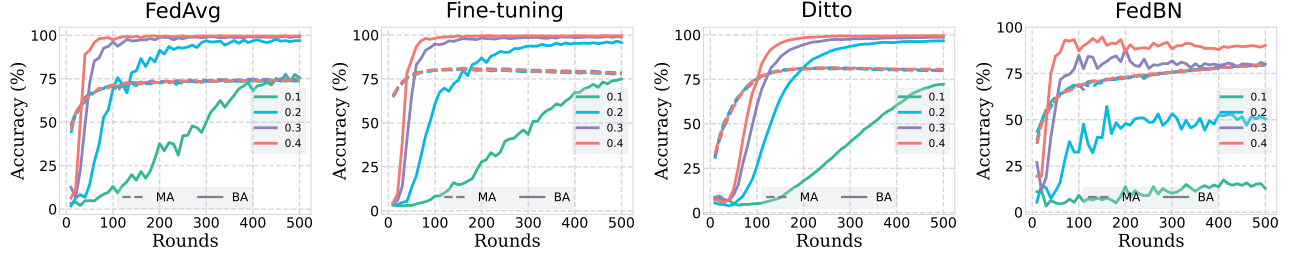


Figure 2: Impact of backdoor attack on typical pFL approaches with varying $\delta \in \{0.1, 0.2, 0.3, 0.4\}$.

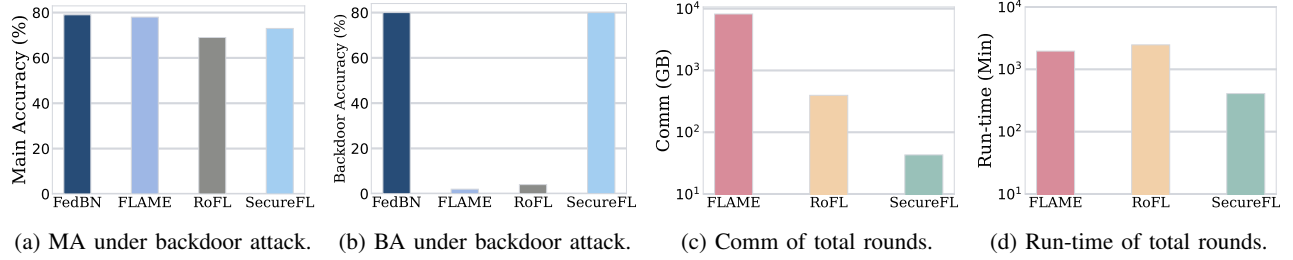


Figure 3: Robustness enhancement and efficiency of different robust secure aggregation methods on FedBN with $\delta = 0.3$.

Adversary Capabilities. In this paper, we define the capabilities for two types of adversaries: 1) Adversary $\mathcal{A}^c$ can compromise up to c users, where $c < \frac{n}{2}$ with n being the total number of clients, but cannot control the remaining benign clients. The malicious clients initiate a backdoor attack by uploading poisoned updates to inject a backdoor trigger into the global model; 2) Adversary $\mathcal{A}^s$ can compromise at most one server, and does not collude with clients, honestly running the aggregation protocol and launching inference attacks to infer private information $\mathcal{D}_i$ from the model updates uploaded by the target client (semi-honest setting).

3. Analysis and Motivation

In this section, we evaluate the robustness of pFL approaches under backdoor attacks, as well as the robustness enhancement of pFL brought by robust secure aggregation methods. We also analyze the efficiency of existing robust secure aggregation methods. Based on the experimental results, we present the motivations of this work.

3.1. Experimental Setup

Dataset and Model. We select the CIFAR-10 [28] image dataset and simulate the Non-IID setting by distributing it among 100 clients using Dirichlet allocation with a concentration factor $\alpha = 0.5$ following [45]. Meanwhile, we employ ResNet-20 [26] for the classification task.

Backdoor Attack Setting. The malicious clients launch the continuous *Badnet Attack*, using grid pattern as trigger to poison half of the training samples, aiming to classify the ‘airplane’ label in CIFAR-10 as ‘ship’. Meanwhile, we use

poisoned model rate $\delta = \frac{m}{n}$ to indicate the ratio of malicious clients m to all participating clients n in the aggregation.

Evaluation Metrics. For robustness evaluation, we consider two metrics: Main Task Accuracy (MA) and Backdoor Task Accuracy (BA). Here, MA denotes the classification accuracy of the model on clean samples, while BA represents the classification accuracy of the model on poisoned samples. The goal of the backdoor attack is to maximize BA while maintaining a high MA of the global model. We evaluate average MA and BA respectively on all clients’ test dataset and the test dataset excluding malicious clients. Each experiment is conducted with 5 times and records the average.

3.2. Impact of Backdoor Attack on pFL

To assess the robustness of pFL schemes against backdoor attacks, we select representative methods Fine-tuning [34], Ditto [32], and FedBN [33] from each classification in Section 2.1 and perform robustness evaluation with FedAvg [37] as the baseline. The training consists of 500 rounds in total, and each round the servers select 10% of the 100 clients to participate in the aggregation. During this process, malicious clients consistently launch backdoor attacks, striving to maximize BA on crafted poisoned samples while maintaining high MA on clean samples. As shown in Figure 2, we measure the MA and BA of different schemes under poisoned model rate $\delta \in \{0.1, 0.2, 0.3, 0.4\}$, and observe that:

O1: We observe that backdoor attacks successfully demonstrate their stealth across various schemes, maintaining high MA on clean samples while continuously increasing BA on poisoned samples. Furthermore, we can observe that both Fine-tuning and the Regularization-based scheme

Ditto are susceptible to backdoor attacks, achieving a high BA of 72-75% at $\delta = 0.1$, which is no different from FedAvg. In contrast, the Parameter-decoupling scheme FedBN manages to control BA of around 20% at $\delta = 0.1$, effectively limiting the convergence of the backdoor task. However, as the number of attackers increases, BA also rises. When $\delta = 0.4$, BA quickly reaches around 85%.

To evaluate the backdoor-resistant capability enhancement of pFL brought by existing robust secure aggregation methods, we conduct experiments of state-of-the-art robust secure aggregation approaches: FLAME (cluster filtering), RoFL (L_2 -norm bound), and SecureFL (similarity detection) on FedBN in the Non-IID setting. In Figure 3a-3b, with $\delta = 0.3$, we measure the MA and BA of training process of FedBN based on different robust secure aggregation methods against *Badnet Attack*. In Figure 3c-3d, we assess the communication cost and run-time of completing all rounds aggregation for each robust secure aggregation scheme in both LAN and WAN setting. Following [15], the client to server (C-S) connection is over 50 Mbps WAN with 50 ms round trip time (rtt) and server to server (S-S) connection is over 10 Gbps LAN with 0.2 ms rtt. Then, we observe that:

O2: As shown in Figure 3a-3b, both FLAME (cluster filtering) and RoFL (L_2 -norm bound) can effectively reduce BA, but RoFL leads to a significant decrease in MA by 10% compared with FedBN. In contrast, SecureFL (similarity detection) cannot effectively defend against backdoor attacks in Non-IID scenarios, resulting in the BA up to 80%.

O3: In Figure 3c-3d, FLAME and RoFL face a large communication overhead, resulting in a long run-time of end-to-end aggregation on total 500 training rounds. While, the detection algorithm of SecureFL is lighter, thus the aggregation run-time is faster.

3.3. Motivations

Based on the observations above, we have summarized the following motivations:

M1: pFL is vulnerable to backdoor attacks. As Observation 1, we validate that pFL based on the Parameter-decoupling method can defend against a limited number of malicious attackers. However, we also observe that as the number of malicious attackers increases, the backdoor task can quickly converge on existing pFL schemes. This implies that as poisoned updates increase, the global model used for generalization is rapidly tainted with backdoor trigger, and local personalized training fails to effectively remove the poisoned features.

M2: pFL lacks a practically robust secure aggregation method. Through Observation 2 and 3, we find that both FLAME and RoFL are effective in defending against backdoor attacks, but RoFL leads to a significant decrease in MA. By contrast, SecureFL, utilizing the fixed server-side validation dataset for similarity detection, fails to effectively distinguish between benign and poisoned updates due to significant differences between updates in the Non-IID setting. However, the use of complex cryptographic tools and detection algorithms by RoFL and FLAME results in

inefficient secure aggregation. The primary communication in RoFL originates from the transmission HE ciphertexts during the C-S phase, and the creation and verification of L_2 -norm proofs entail substantial time overhead, thus resulting in low aggregation efficiency. FLAME incurs significant communication cost due to the high complexity of its clustering detection. While the detection protocol operates in the S-S phase, the 2PC protocol executed in the LAN environment mitigates some aggregation pressure but also leads to prolonged run-time. Conversely, SecureFL adopts a linear detection method from FLTrust, making the aggregation process more faster. Overall, current pFL lacks robust secure aggregation strategy that balance robustness and aggregation efficiency.

Therefore, we require an efficient detection method for pFL that can resist backdoor attacks in Non-IID scenarios, while design a lightweight robust secure aggregation for fast aggregation.

4. FLIGHT Design

In this section, we first identify the goals of designing a robust secure aggregation that balances backdoor-resistant capability and aggregation efficiency. To achieve these goals, we provide a detailed design of FLIGHT in terms of security and privacy.

4.1. Goals and Solutions

In the FL framework, client $C_i, i \in [1, n]$ minimizes the local loss function using their local data on the global model $\mathbf{w}_g^{t-1}$ of last training round $t-1$, obtaining the local model:

$$\mathbf{w}_i^t = \mathbf{w}_g^{t-1} - \eta_i \nabla \mathcal{L}_i(\mathbf{w}_g^{t-1}) \quad (4)$$

Here, η_i represents the local learning rate. Subsequently, C_i uploads the model updates $\Delta \mathbf{w}_i^t = \mathbf{w}_i^t - \mathbf{w}_g^{t-1}$ to the server. The server aggregates the updates with the weight factor γ_i to minimize the global task loss $\mathcal{L}_g(\mathbf{w}_g^{t-1}) = \sum_i \gamma_i \mathcal{L}_i(\mathbf{w}_g^{t-1})$, moving the global model $\mathbf{w}_g^t$ towards the direction of the local models $\mathbf{w}_i^t$ as:

$$\begin{aligned} \mathbf{w}_g^t &= \mathbf{w}_g^{t-1} - \nabla \mathcal{L}_g(\mathbf{w}_g^{t-1}) \\ &= \mathbf{w}_g^{t-1} - \sum_i \gamma_i (\mathbf{w}_g^{t-1} - \mathbf{w}_i^t) \end{aligned} \quad (5)$$

However, the malicious clients aim to replace the global model by crafting poisoned updates $\Delta \mathbf{w}_m^t = \mathbf{w}_g^{t-1} - \mathbf{w}_m^t, m \in [\text{malicious clients}]$. This move shifts the global parameter $\mathbf{w}_g^t$ towards $\mathbf{w}_m^t$, minimizing both the main task and the backdoor task losses:

$$\mathbf{w}_g^t = \mathbf{w}_g^{t-1} - \gamma_m (\mathbf{w}_g^{t-1} - \mathbf{w}_m^t) \quad (6)$$

Many works assume that the poisoned updates $\Delta \mathbf{w}_m^t$ differ significantly from benign ones and exclude $\Delta \mathbf{w}_m^t$ by assigning them smaller γ_m . However, in the Non-IID setting, due to the heterogeneity of local data among different C_i , there exists a large difference between local models $\mathbf{w}_i^t$, which

diminishes the distinction between benign and poisoned updates. This poses a challenge for defense methods. In Section 3.2, we identify that current robust secure aggregation techniques face challenges in balancing robustness and efficiency:

- High efficiency but poor robustness, typically involving low-complexity linear defense operations (SecureFL).
- Low efficiency but strong robustness, typically employing high-complexity non-linear defense operations (FLAME).

Therefore, to enhance both robustness and efficiency, our goal is to design a lightweight backdoor detection mechanism that achieves backdoor defense capabilities similar to FLAME while maintaining the linear complexity of SecureFL. This would reduce communication cost in MPC, leading to efficient and robust secure aggregation.

To achieve this, we design FLIGHT, a solution that balances robustness and efficiency. FLIGHT implements a two-stage **Personalized Defense Mechanism** to counter backdoor attacks and utilizes MPC to achieve **Efficient Robust Secure Aggregation** while simultaneously defending against inference attacks:

Personalized Defense Mechanism (Security): We design a two-stage defense strategy to achieve an efficient and robust defense mechanism. In the Stage1, we utilize a non-linear clustering algorithm for a few rounds to identify initial benign updates. In the Stage2, we use the benign updates from Stage1 to construct personalized validation models for each C_i . The difference between the actual model and its validation model is then calculated to obtain a personalized score with linear complexity, which is communication-friendly for MPC. Subsequently, the scores are employed to perform weighted aggregation to eliminate poisoning updates and dynamically upgrade each validation model for the next round of aggregation, adapting to the Non-IID setting.

Efficient Robust Secure Aggregation (Privacy): Leveraging the advantages of the two-stage defense mechanism, we design a series of efficient MPC protocols to achieve robust secure aggregation while simultaneously defending against both poisoning and inference attacks. These protocols require invoking a communication-intensive clustering protocol only a few times during Stage1. For the majority of the subsequent training rounds, we apply the linear detection method from Stage2, using a series of communication-efficient 2PC protocols, which significantly reduces the end-to-end training communication cost.

Next, we will systematically explain the implementation details of Personalized Defense Mechanism and Efficient Robust Secure Aggregation, with the notations used in this work presented in Table 3 of Appendix A.

4.2. Personalized Defense Mechanism

In order to achieve a lightweight and effective backdoor defense in the Non-IID setting, FLIGHT introduces a two-

stage personalized defense mechanism (shown in Algorithm 1), which is friendly to MPC communication:

Algorithm 1: FLIGHT personalized defense mechanism

Input: Initial global model $\mathbf{w}_g^0$.
Output: Global model $\mathbf{w}_g^R$ after R training rounds.

- 1 **Local Training:**
- 2 **for** each training round $t \in [1, R]$ **and** client $i \in [1, n]$ **do**
- 3 $\Delta \mathbf{w}_i^t = \text{LocalTrain}(\mathbf{w}_i^{t-1})$, $l_i = \|\Delta \mathbf{w}_i^t\|_2$.
- 4 **end**
- 5 **FLIGHT-Stage1:**
- 6 **for** each training round $t \in [1, K]$ **do**
- 7 $[\text{BenignList}] = \text{Cluster}(\Delta \mathbf{w}_i^t, i \in [1, n])$.
- 8 **end**
- 9 Initialize validation model for C_i :
 $\mathbf{p}_i^t = \text{Mean}(\Delta \mathbf{w}_j^t), j \in [\text{BenignList}]$.
- 10 **FLIGHT-Stage2:**
- 11 **for** each training round $t \in [K+1, R]$ **do**
- 12 § **Personalized score:**
- 13 $\text{cos}_i =$
 $\quad \text{CosineSimilarity}(\Delta \mathbf{w}_i^t, \mathbf{p}_i^t)$. $\triangleright$ cosine similarity
- 14 $T_i = (\text{Mean}(l_{i \in [1, n]}) / l_i)^{\text{sgn}(\text{cos}_i)}$. $\triangleright$ norm temperature
- 15 $ps_i = \text{cos}_i \cdot T_i$. $\triangleright$ personalized score
- 16 $\gamma_i = \text{Softmax}(ps_i)$. $\triangleright$ weight factor
- 17 $\mathbf{w}_g^t = \mathbf{w}_g^{t-1} - \sum_i \gamma_i \cdot \Delta \mathbf{w}_i^t$. $\triangleright$ weight aggregation
- 18 § **Personalized validation model:**
- 19 $\eta_i = \text{Select}(\gamma_i)$. $\triangleright$ learning rate
- 20 $\mathbf{p}_i^{t+1} =$
 $\quad \mathbf{p}_i^t - \eta_i(\mathbf{p}_i^t - \Delta \mathbf{w}_i^t)$. $\triangleright$ upgrade validation model
- 21 **end**
- 22 **return** $\mathbf{w}_g^R$.

FLIGHT-Stage1. To improve the accuracy of identifying poisoning updates, we first employ a clustering algorithm from FLAME to select out candidate benign updates. This phase is executed only during the initial few K rounds with the aim of identifying the direction of benign updates. Subsequently, we aggregate these selected benign updates to assign an initial validation model $\mathbf{p}_i^t = \text{Mean}(\Delta \mathbf{w}_j^t), j \in [\text{benign updates}]$ to each C_i .

FLIGHT-Stage2. To further reduce the complexity of the backdoor detection algorithm during the subsequent numerous training rounds, we utilize personalized scores and personalized validation models to implement a detection algorithm that scales linearly with the number of clients n .

(1) *Personalized Score.* Upon receiving the update $\Delta \mathbf{w}_i^t$ from C_i , the server determines whether $\Delta \mathbf{w}_i^t$ is poisoned by checking the inconsistency between $\Delta \mathbf{w}_i^t$ and $\mathbf{p}_i^t$. We define personalized score to describe the inconsistency. Based on this score, we perform weight aggregation and dynamic upgrading of $\mathbf{p}_i^t$. Following the analysis of [9], [41], [55], poisoned updates often change direction to guide the global model towards the target backdoor task. Hence, we first employ cosine similarity to measure the angular differences of direction between each $\Delta \mathbf{w}_i^t$ and its validation model $\mathbf{p}_i^t$ as:

$$\text{cos}_i = \frac{\Delta \mathbf{w}_i^t \cdot \mathbf{p}_i^t}{\|\Delta \mathbf{w}_i^t\|_2 \cdot \|\mathbf{p}_i^t\|_2} \quad (7)$$

Additionally, malicious clients also amplify the weight of

poisoned $\Delta \mathbf{w}_i^t$ to maximize the attack impact, resulting in the L_2 -norm $l_i = \|\Delta \mathbf{w}_i^t\|_2$ being greater than the benign ones. Therefore, we design norm temperature T_i to assign the potential poisoned updates corresponding penalties based on the magnitude of l_i as:

$$T_i = \left(\frac{\text{Mean}(l_{i \in [1, n]})}{l_i} \right)^{\text{sgn}(\cos_i)} \quad (8)$$

When the cosine similarity $\cos_i < 0$, the norm temperature $T_i = l_i / \text{Mean}(l_{i \in [1, n]})$, which is positively correlated with the norm size l_i . When $\cos_i \geq 0$, norm size l_i acts negatively on the $T_i = \text{Mean}(l_{i \in [1, n]}) / l_i$. Furthermore, we combine cosine similarity with norm temperature for C_i to compute personalized scores ps_i as:

$$ps_i = \cos_i \cdot T_i \quad (9)$$

We make malicious clients obtain smaller ps_i , while benign clients receive larger ps_i . In other words, when the direction of the update is inconsistent with its validation model, we judge it with high probability as a poisoned update, assigning $ps_i < 0$ and further decreasing the score through T_i positively correlated with l_i . Conversely, we assign $ps_i \geq 0$, and to avoid missing poisoned updates that align with the direction of its validation model, we enable T_i negatively correlated with l_i to reduce the impact of these updates. Meanwhile, the $\text{Mean}(l_{i \in [1, n]})$ restricts the significant gaps between each score, thus avoiding the jiggling that may occur during training.

Next, to better converge the global model, we regularize personalized score ps_i into weights γ_i using Softmax function as:

$$\gamma_i = \text{Softmax}(ps_i) \quad (10)$$

Finally, we perform weight aggregation to obtain: $\mathbf{w}_g^t = \mathbf{w}_g^{t-1} - \sum_i^n \gamma_i \cdot \Delta \mathbf{w}_i^t$.

(2) *Personalized Validation Model.* Furthermore, after each training round t , we utilize historical updates $\Delta \mathbf{w}_i^t$ of C_i to upgrade the validation model $\mathbf{p}_i^{t+1} = \mathbf{p}_i^t - \eta_i (\mathbf{p}_i^t - \Delta \mathbf{w}_i^t)$ for the next-round aggregation, dynamically adapting to changes in local updates during training. Inspired by [42], the learning rate η_i is determined by the Select function based on the magnitude relationship between weight γ_i and threshold θ_p as:

$$\text{Select}(\gamma_i) = \begin{cases} \eta_i = 0 & \text{if } \gamma_i < \theta_p \\ \eta_i = \eta & \text{if } \gamma_i \geq \theta_p \end{cases} \quad (11)$$

Where γ_i , calculated in Eq. 10, represents the aggregation weight of C_i . When $\gamma_i < \theta_p$, we classify the update from malicious client and return $\eta_i = 0$, indicating that model $\mathbf{p}_i^{t+1}$ remains unchanged to prevent learning the backdoor task. Conversely, when $\gamma_i \geq \theta_p$, we return $\eta_i = \eta$, allowing model $\mathbf{p}_i^{t+1}$ to move in the direction of benign update $\Delta \mathbf{w}_i^t$ as usual.

This approach avoids misclassifying poisoned updates in the Non-IID setting where local models are widely dispersed, and the injective detection method prevent using high complexity clustering algorithm in all training rounds.

Instead, clustering is only required during the initial few K rounds to initialize the validation model $\mathbf{p}_i^t$ for each C_i .

4.3. Efficient Robust Secure Aggregation

In this work, we employ two servers $S_j, j \in [1, 2]$ to achieve robust secure aggregation based on secure two-party computation (2PC) [13], [39]. We implement additive secret sharing (denoted as $[\cdot]$ -sharing), where a secret value $x \in \mathbb{Z}_{2^\ell}$ is shared as two random values $[x]_1, [x]_2 \in \mathbb{Z}_{2^\ell}$ with $x = [x]_1 + [x]_2 \pmod{2^\ell}$. When $\ell > 1$, it supports arithmetic operations (e.g., $+$, $-$, and $\cdot$), denoted as arithmetic sharing. Specifically, when $\ell = 1$, this type is referred to boolean sharing and use notation $[\cdot]^B$, where $+$, $-$ and $\cdot$ are respectively replaced by bit-wise $\oplus$ and $\wedge$. Without special declaration, we compute in $\mathbb{Z}_{2^\ell}$ and omit $\pmod{2^\ell}$ for brevity.

(1) *Sharing and Reconstruction.* Π_{SS} allows client share the secret x to S_j : client selects $[x]_1 \in_R \mathbb{Z}_{2^\ell}$, computes $[x]_2 = x - [x]_1$, and sends $[x]_j$ to S_j . While, Π_{Rec} is used to reconstruct the secret x : S_j sends $[x]_j$ to S_{j+1} , then S_j can reconstruct $x = [x]_1 + [x]_2$.

(2) *Addition.* Given public constants μ, v, λ and secret sharing $[x], [y]$, the sharing $[z] = \mu[x] + v[y] + \lambda$ can be computed locally by S_1, S_2 as $[z]_1 = \mu[x]_1 + v[y]_1 + \lambda$ and $[z]_2 = \mu[x]_2 + v[y]_2$.

(3) *Multiplication.* Given secret sharing $[x], [y]$, the sharing $[z] = [x] \cdot [y]$ can be achieved by Π_{Mul} . In the Π_{Mul} , we use pre-computed beaver triple $([a], [b], [c])$, where $c = a \cdot b$. Then, S_j computes $[e]_j = [x]_j - [a]_j$ and $[f]_j = [y]_j - [b]_j$, both party executes $e = \Pi_{\text{Rec}}([e]_j)$ and $f = \Pi_{\text{Rec}}([f]_j)$, finally set $[z]_j = -jef + f[x]_j + e[y]_j + [c]_j$. Meanwhile, we also define Π_{MulAB} to achieve arithmetic-boolean multiplication $[z] = [x] \cdot [y]^B$.

(4) *Underlying protocols.* Besides above basic operations, we invoke non-linear protocols reciprocal (Π_{Recip}), compare (Π_{Cmp}), and maximum (Π_{Max}) from the state-of-the-art work [46] in a black-box manner. The inputs and outputs of these protocols are as follows:

- $[z] = \Pi_{\text{Recip}}([x])$ s.t. $z = 1/x$.
- $[z] = \Pi_{\text{Cmp}}([x])$ s.t. $z = 1\{x \geq 0\}$.
- $[z] = \Pi_{\text{Max}}([x_{i \in [1, n]}])$ s.t. $z = \text{Maximum}(x_{i \in [1, n]})$.

Building on the foundational MPC protocols, we have developed the FLIGHT robust secure aggregation framework, as illustrated in Figure 4. Next, we will progressively detail the execution steps of the framework:

For FLIGHT-Stage1, ① we can directly use the 2PC clustering protocol Π_{Cluster} provided by FLAME. Although it has a high communication cost, it is only executed during the initial few rounds, thus not significantly impacting the overall efficiency of FLIGHT.

For FLIGHT-Stage2, we implement communication-efficient 2PC protocols for linear detection method: cosine similarity (Π_{Cos}), norm temperature (Π_{NT}), and softmax (Π_{Softmax}) based on the underlying protocols for securely computing personalized weight factor and upgrading validation model:

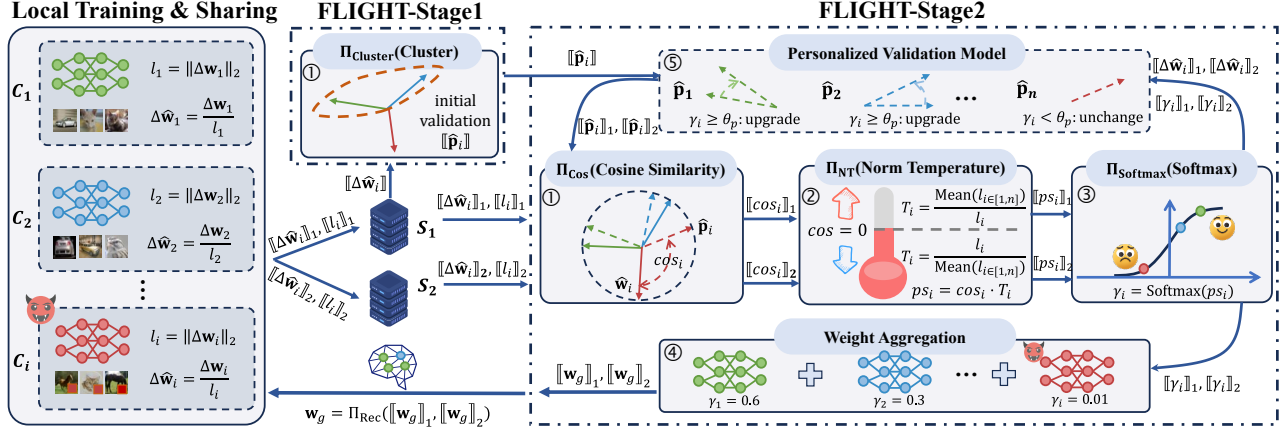


Figure 4: Framework of FLIGHT.

Upload Setting. During the model updates uploading phase, we further reduce the servers' non-linear MPC operations by assisting client C_i in computing the model update norm l_i locally and normalizing the updates: C_i undergoes local training to obtain the t round model update $\Delta \mathbf{w}_i^t$, computes the norm of the update $l_i = \|\Delta \mathbf{w}_i^t\|_2$, and then normalizes update $\Delta \hat{\mathbf{w}}_i^t = \Delta \mathbf{w}_i^t / l_i$, finally secret-shares $(\llbracket \Delta \hat{\mathbf{w}}_i^t \rrbracket_j, \llbracket l_i \rrbracket_j) = \Pi_{SS}(\Delta \hat{\mathbf{w}}_i^t, l_i)$ and uploads to the servers $S_j, j \in \{1, 2\}$.

① **Cosine Similarity.** To securely compute the angular change between the update $\llbracket \Delta \hat{\mathbf{w}}_i^t \rrbracket$ and its validation update $\llbracket \hat{\mathbf{p}}_i \rrbracket$, we design the secure cosine similarity protocol Π_{Cos} . Benefit from the update normalization, the servers can directly compute the cosine similarity $\llbracket cos_i \rrbracket$ using the secure multiplication protocol Π_{Mul} : $\llbracket cos_i \rrbracket = \Pi_{Cos}(\llbracket \Delta \hat{\mathbf{w}}_i^t / l_i \rrbracket, \llbracket \hat{\mathbf{p}}_i^t / l_i \rrbracket) = \Pi_{Mul}(\llbracket \Delta \hat{\mathbf{w}}_i^t \rrbracket, \llbracket \hat{\mathbf{p}}_i^t \rrbracket)$.

② **Norm Temperature.** After obtaining $\llbracket cos_i \rrbracket$, we define Π_{NT} to compute the norm temperature under secret-sharing. The norm temperature in FLIGHT is defined as: $(\text{Mean}(l_{i \in [1, n]}) / l_i)^{\text{sgn}(cos_i)}$, where $l_i / \text{Mean}(l_{i \in [1, n]})$ is obtained when $cos_i < 0$, and conversely if $cos_i \geq 0$, we get $\text{Mean}(l_{i \in [1, n]}) / l_i$. We first invoke Π_{Mean} , Π_{Recip} to compute the mean $\llbracket m \rrbracket$ of norms $\llbracket l_{i \in [1, n]} \rrbracket$ and its reciprocal $1/\llbracket m \rrbracket$, then calculate the reciprocal $1/\llbracket l_i \rrbracket$ for each client. Subsequently, we compute the candidate values $\llbracket l_i / m \rrbracket = \Pi_{Mul}(\llbracket l_i \rrbracket, \llbracket 1/m \rrbracket)$ and $\llbracket m / l_i \rrbracket = \Pi_{Mul}(\llbracket m \rrbracket, \llbracket 1/l_i \rrbracket)$. Next, for the sign extraction of $\llbracket cos_i \rrbracket$, we invoke Π_{Cmp} , which returns a boolean judgment $\llbracket b_i \rrbracket^B = 1\{cos_i \geq 0\}$. The key challenge lies in how to choose between $\llbracket l_i / m \rrbracket$ and $\llbracket m / l_i \rrbracket$ drew on the judgment $\llbracket b_i \rrbracket^B$. We utilize Π_{MulAB} to achieve arithmetic-boolean multiplication $(\llbracket m / l_i \rrbracket - \llbracket l_i / m \rrbracket) \cdot \llbracket b_i \rrbracket^B + \llbracket l_i / m \rrbracket$, where return $\llbracket l_i / m \rrbracket$ when $\llbracket b_i \rrbracket^B = 0$ and $\llbracket m / l_i \rrbracket$ when $\llbracket b_i \rrbracket^B = 1$. Based on above perspective, Π_{NT} is constructed in Protocol 1.

③ **Softmax.** After receiving $\llbracket cos_i \rrbracket, \llbracket T_i \rrbracket$, we can easily compute the personalized scores $\llbracket ps_i \rrbracket = \Pi_{Mul}(\llbracket cos_i \rrbracket, \llbracket T_i \rrbracket)$. To perform the Softmax operation on $\llbracket ps_i \rrbracket$, we define $\Pi_{Softmax}$, which first computes the maximum value $\llbracket ps_{max} \rrbracket = \Pi_{Max}(\llbracket ps_{i \in [1, n]} \rrbracket)$ satisfying $ps_i - ps_{max} \leq 0$, and then

proceeds to calculate:

$$\Pi_{Softmax}(ps_i) = \frac{\text{negExp}(\llbracket ps_i \rrbracket - \llbracket ps_{max} \rrbracket)}{\sum_{i=1}^n \text{negExp}(\llbracket ps_i \rrbracket - \llbracket ps_{max} \rrbracket)} \quad (12)$$

The state-of-the-art approaches [46], [24] utilize lookup-table (LUT)-based protocols to compute the negExp function. However, the communication efficiency is lower when the input bit length is large. To further reduce communication overhead, we adopt the 2Quad method [31] and combine it with piecewise function to approximate the negExp function as follows:

$$\text{negExp}(x) \approx \begin{cases} 0 & \text{if } x < \theta_{exp} \\ (1+x)^2 & \text{if } x \geq \theta_{exp} \end{cases} \quad (13)$$

We perform secure comparisons between $\llbracket ps_i \rrbracket$ and threshold θ_{exp} using Π_{Cmp} , followed by square approximation through $\llbracket z_i^2 \rrbracket = \Pi_{Mul}(\llbracket z_i \rrbracket, \llbracket z_i \rrbracket)$, where $z_i = 1 + (ps_i - ps_{max})$. To further reduce non-linear operations, we sum up $\llbracket z_{sum}^2 \rrbracket = \sum_{i=1}^n (\llbracket z_i^2 \rrbracket)$ and take its reciprocal to obtain $\llbracket 1/z_{sum}^2 \rrbracket = \Pi_{Recip}(\llbracket z_{sum}^2 \rrbracket)$. Subsequently, we perform $\Pi_{Mul}(\llbracket z_i^2 \rrbracket, \llbracket 1/z_{sum}^2 \rrbracket)$ operations n times to acquire weights $\llbracket \gamma_i \rrbracket$, avoiding the need for n times non-linear division $\text{Div}(\llbracket z_i^2 \rrbracket, \llbracket z_{sum}^2 \rrbracket)$. The detailed definition of $\Pi_{Softmax}$ is provided in Protocol 2.

④ **Weight Aggregation & ⑤ Validation Model Upgrade.** To perform weight aggregation correctly, the servers first need to restore the normalized model $\llbracket \Delta \hat{\mathbf{w}}_i^t \rrbracket$ back to $\llbracket \Delta \mathbf{w}_i^t \rrbracket$ by multiplying it with its norm $\llbracket l_i \rrbracket$, where $\Delta \mathbf{w}_i^t \in \mathbb{R}^{1 \times d}$. Then, it should be multiplied with the weight factor $\llbracket \gamma_i \rrbracket$, requiring $2 \times d$ times Π_{Mul} for each C_i . To reduce the number of calls to Π_{Mul} , we first compute the product of weight and norm $\llbracket l_i \cdot \gamma_i \rrbracket = \Pi_{Mul}(\llbracket l_i \rrbracket, \llbracket \gamma_i \rrbracket)$, then multiply it with the normalized model $\llbracket \Delta \hat{\mathbf{w}}_i^t \rrbracket$ and aggregate to obtain the global model $\llbracket \mathbf{w}_g^t \rrbracket = \llbracket \mathbf{w}_g^{t-1} \rrbracket - \sum_{i=1}^n \Pi_{Mul}(\llbracket l_i \cdot \gamma_i \rrbracket, \llbracket \Delta \hat{\mathbf{w}}_i^t \rrbracket)$, achieving $d+1$ times Π_{Mul} for each C_i . Finally, the server invoke $\Pi_{Rec}(\llbracket \mathbf{w}_g^t \rrbracket)$ to reconstruct the aggregation result $\mathbf{w}_g^t$ and send it back to each C_i .

Furthermore, for each validation model of C_i , we only need to upgrade its normalized model $\llbracket \hat{\mathbf{p}} \rrbracket$. Let the

weights $\llbracket \gamma_i \rrbracket$ be compared with the threshold θ_p to get $\llbracket b_i \rrbracket^B = \Pi_{\text{Cmp}}(\llbracket \gamma_i \rrbracket - \theta_p)$, and execute $\llbracket \hat{\mathbf{p}}_i^t \rrbracket = \llbracket \hat{\mathbf{p}}_i^{t-1} \rrbracket - \Pi_{\text{Mul}}(\Pi_{\text{MulAB}}(\eta_i, \llbracket b_i \rrbracket), \llbracket \hat{\mathbf{p}}_i^{t-1} \rrbracket - \llbracket \Delta \hat{\mathbf{w}}_i^t \rrbracket)$ to complete the upgrade.

5. Theoretical Analysis

5.1. Security and Correctness Analysis.

Theorem 1. *Given the ideal protocols Π_{Mul} , Π_{MulAB} , Π_{Mean} , Π_{Recip} , Π_{Cmp} , and Π_{Max} defined in [46], FLIGHT securely realize the protocols Π_{Cos} , Π_{NT} , and Π_{Softmax} against the semi-honest $\mathcal{A}^s$, ensuring $\mathcal{A}^s$ cannot infer anything beyond the aggregation result $\mathbf{w}_g$.*

Proof. We provide the proof that relies on the stand-alone model under sequential composition in Appendix C.1. $\square$

Theorem 2. *The ciphertext operations in FLIGHT correctly execute secure aggregation without compromising backdoor-resistant effectiveness.*

Proof. We provide the proof of FLIGHT’s correctness in Appendix C.2. $\square$

6. Evaluation

In this section, we conduct extensive experiments to compare the robustness against backdoor attacks and aggregation efficiency of our approach FLIGHT with existing robust secure aggregation methods FLAME [41], RoFL [35], and SecureFL [25].

Experimental Environment. We implement FLIGHT in Python3 and C++. We achieve GPU-based ciphertext aggregation based on FederatedScope [54] and Piranha [51] while running our experiments on Intel(R) Xeon(R) Silver 4314 CPU @ 2.40 GHz with 512 GB RAM and Nvidia V100 with 40 GB RAM. We consider two types of network environments LAN and WAN. The client to server (C-S) connection is over 50 Mbps WAN with 50 ms round trip time (rtt) and server to server (S-S) connection is over 10 Gbps LAN with 0.2 ms rtt.

TABLE 1: Training Tasks.

	MNIST	Fashion	CIFAR-10S	CIFAR-10L
Dataset	MNIST [30]	F-MNIST [52]	CIFAR-10 [28]	CIFAR-10 [28]
Model	LeNet [29]	LeNet [29]	ResNet-20 [26]	ResNet-20L [26]
Params	62K	62K	273K	1.3M

Training Task Baselines. For a fair comparison, we follow approaches [41], [35], [25] by selecting public benchmark datasets and network models, which together form the four image classification tasks listed in Table 1. Both CIFAR-10S and CIFAR-10L use the CIFAR-10 dataset but employ different neural network models with varying parameter sizes. The increase in parameter size directly impacts the efficiency of encrypted aggregation, though its effect on robustness is less significant. Therefore, CIFAR-10S is primarily used

for robustness evaluation, while CIFAR-10L is utilized for efficiency testing under larger-scale parameters.

Training Details. We conduct robustness testing using three mainstream backdoor attack approaches: *Badnet*, *Scaling Attack*, and *DBA* following the backdoor setting and metrics described in Section 3.1, and simulate the Non-IID setting by distributing dataset among all clients using Dirichlet factor $\alpha = 0.5$ following [45]. We employ Main Task Accuracy (MA) to denote the classification accuracy of the model on clean samples and Backdoor Task Accuracy (BA) to represent the classification accuracy of the model on poisoned samples, where the goal of the backdoor attack is to maximize BA while maintaining a high MA of the global model.

6.1. Robustness Analysis

In this section, we analyze the robustness of backdoor defense mechanism proposed by FLIGHT under different types of backdoor attacks and varying poisoned model rate δ . The training consists of 500 rounds in total, and each round the servers select 10% of the 100 clients to participate in the aggregation.

Impact of Varying Backdoor Attacks. We choose backdoor attacks: *Badnet*, *Scaling Attack*, and *DBA* to evaluate whether FLIGHT can assist the pFL approach, FedBN, in mitigating backdoor attacks from malicious clients. The defense objective is to decrease BA on poisoned samples while maintaining normal convergence of MA. Table 2 shows the performance of different training tasks when the $\delta = 0.3$. We observe that FLIGHT achieves defense results similar to the state-of-the-art FLAME, limiting the BA to 0.01-5.96% while maintaining a high level of MA as 78.96-99.62%. In contrast, SecureFL, based on the fixed server-side validation dataset, fails to effectively remove poisoned updates in Non-IID scenarios, resulting in high BA as 80.26-99.25%. Additionally, it filters out some benign updates, leading to a slight decrease in MA compared to FedBN by 2.79-4.83%. RoFL employs L_2 -norm bound to limit the injection of backdoor tasks, especially against *Scaling Attack*. However, the smaller bound also restricts the impact of benign updates, resulting in a lower MA level and a significant decrease of 5.09-11.45% compared to FedBN.

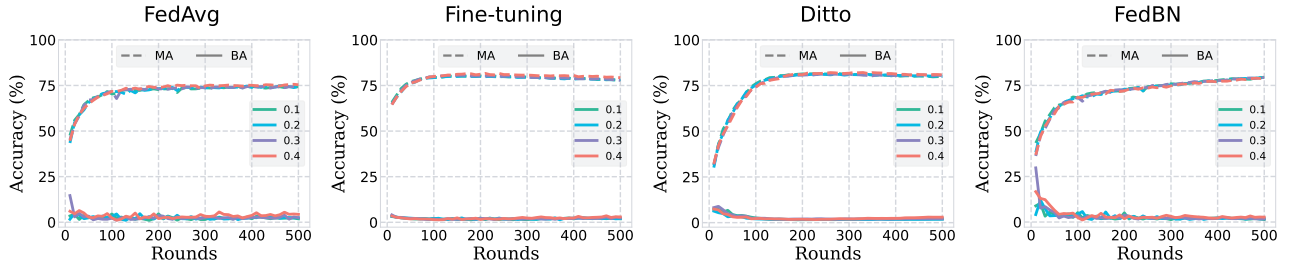
Impact of Varying δ . We verify in Section 3.2 that as the δ increases, the backdoor task converges faster on the pFL. To evaluate the backdoor-resistant effectiveness of FLIGHT under different δ , Figure 5 shows the robustness improvement of FLIGHT under *Badnet Attack* with $\delta = \{0.1, 0.2, 0.3, 0.4\}$ for FedAvg and different types of pFL schemes during CIFAR-10S training mission: we can observe that FLIGHT provides a strong backdoor defense for various pFL schemes at different δ , limiting BA to a low level of 0.01-0.02%.

6.2. Efficiency Analysis

In this section, we analyze the efficiency of FLIGHT (Stage1 & Stage2) under varying numbers of clients n and

TABLE 2: MA and BA of FLIGHT and previous works under varying backdoor attacks and training tasks with $\delta=0.3$.

Attack	Training Task	FedBN		RoFL		FLAME		SecureFL		FLIGHT	
		MA	BA	MA	BA	MA	BA	MA	BA	MA	BA
Badnet	MNIST	99.73	95.90	94.28	1.09	99.67	0.01	95.59	90.41	99.62	0.01
	Fashion	91.85	90.14	83.76	2.57	90.78	0.01	89.06	82.99	91.79	0.01
	CIFAR-10S	80.29	80.95	73.03	2.93	78.29	0.01	75.46	80.26	79.39	0.02
Scaling Attack	MNIST	99.69	99.36	93.34	0.09	99.60	0.24	96.06	95.17	99.19	0.43
	Fashion	89.30	93.59	81.49	0.32	89.25	0.37	86.33	90.38	90.95	0.58
	CIFAR-10S	80.13	90.27	70.86	0.56	78.03	0.42	75.36	88.92	79.81	0.32
DBA	MNIST	99.02	99.16	91.25	2.04	99.07	1.21	99.15	97.88	99.02	0.99
	Fashion	90.46	99.31	79.01	5.94	89.51	6.02	89.08	99.25	90.55	3.65
	CIFAR-10S	79.56	95.73	69.52	7.10	78.66	5.67	76.94	95.81	78.96	5.96

Figure 5: Robust enhancement brought by FLIGHT on typical pFL approaches under Badnet attack with $\delta \in \{0.1, 0.2, 0.3, 0.4\}$.

different tasks (model parameters d) in completing total 500 training rounds.

Comparison with Previous Works. Figure 6 demonstrates the improvement in total aggregation efficiency of FLIGHT, combining Stage1 and Stage2, compared to previous works (detailed costs are provided in Appendix D.1). Figure 6a and 6b illustrate the communication cost and run-time under different training tasks, we find that FLIGHT incurs a $3\times$ higher communication cost than SecureFL due to the use of the communication-intensive protocol Π_{Cluster} in Stage1. However, it reduces the communication by $3\text{--}67\times$ compared to FLAME and RoFL. This is because we only invoke a limited amount of Π_{Cluster} during the early rounds of training, and implement a communication-efficient 2PC protocol in Stage2. Moreover, our GPU-based ciphertext matrix multiplication significantly reduce the run-time of the 2PC protocols, completing the entire aggregation in only $2\text{--}64\times$ less time. Meanwhile, in Figure 6c and 6d, we observe that as client n increases, the overhead faced by prior works during aggregation become intolerable. FLAME, which uses clustering-based detection algorithm with $O(n^2d)$ complexity, experiences a significant increase in communication cost as n grows. Even though it invokes 2PC-based detection protocol Π_{Cluster} in the S-S phase, it still suffers from long aggregation time. RoFL’s communication cost scales linearly with n , but the time required to generate and verify L_2 -norm proofs also increases with n , leading to low aggregation efficiency. SecureFL’s communication cost and run-time are

linearly related to n but because the detection protocol executes in the C-S phase, its run-time is higher compared to FLIGHT. Compared to SecureFL, FLIGHT incurs a increase in communication cost due to the protocol Π_{Cluster} used in Stage1. However, in a LAN setting, the limited invocation of Π_{Cluster} in Stage1 does not result in significant run-time. Additionally, most of the aggregation operations occur at Stage2 with $O(nd)$ complexity, thereby reduce $3\text{--}36\times$ less run-time in total.

Efficiency of Stage2. Furthermore, we evaluate the efficiency of Stage2 with increasing clients n and model parameters d in Figure 7. Benefit of the backdoor detection protocol with complexity $O(nd)$, the communication cost and run-time scale linearly with d and n .

The overall efficiency improvements of FLIGHT can be attributed to two main factors: 1) Our two-stage personalized defense mechanism is friendly to MPC communication, use less clustering protocols to achieve $O(nd)$ linear detection. 2) We design a series of communication-efficient 2PC protocols and utilize GPU to perform fast ciphertext multiplication. As a result, compared to existing robust secure aggregation methods, FLIGHT requires $2\text{--}64\times$ less run-time to achieve total 500 rounds backdoor-resistant secure aggregation.

6.3. Robustness-Efficiency Trade-off Analysis.

Furthermore, we formalize the trade-off between robustness and efficiency under *Badnet Attack* on MNIST

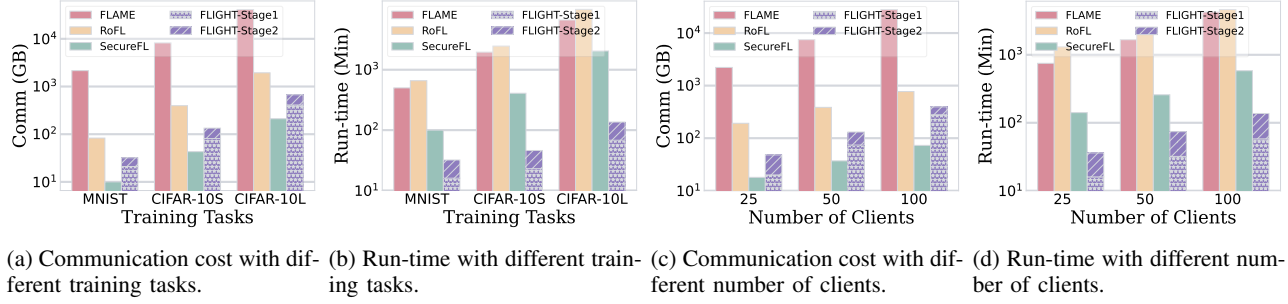


Figure 6: Communication cost (GB) and Run-time (Min) of FLIGHT and previous works. In (a)(b), the number of clients selected to participate in aggregation is fixed as 10. In (c)(d), the training task is fixed as MNIST (62k). Note the y-axis is in log scale.

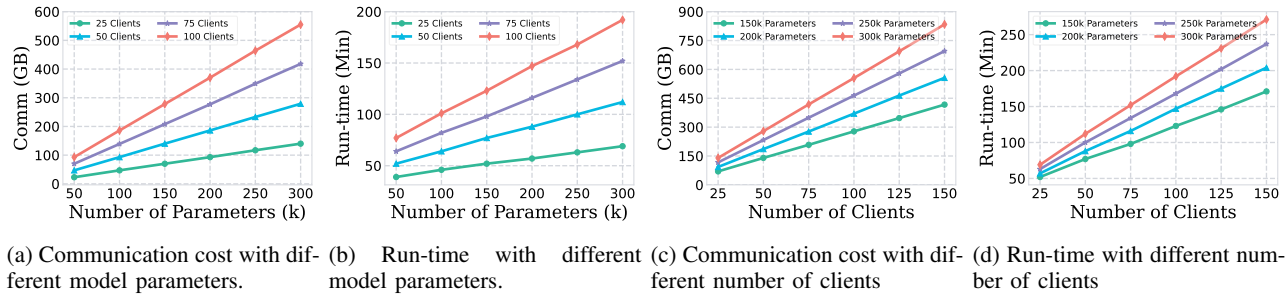


Figure 7: Communication cost (GB) and Run-time (Min) of FLIGHT-Stage2. In (a)(b), different lines represent different number of clients selected to participate in aggregation. In (c)(d), different lines represent different model parameters.

task (Figure 8) across different approaches. We find that FLIGHT, with the shortest aggregation time, effectively limits the convergence of BA on poisoned samples under the Non-IID setting, reducing run-time by approximately 2-17 $\times$ compared to other schemes. In contrast, both FLAME and RoFL demonstrate similar backdoor defense capabilities with lower BA, while RoFL requires a trade-off in the form of reduced MA. However, both methods incur significantly longer run-time. SecureFL’s higher BA illustrates its inability to resist backdoor injections, and its run-time is longer than FLIGHT due to the robust protocol employed during the C-S phase. Additionally, Appendix D.2 demonstrates that in the larger-scale CIFAR-10S and CIFAR-10L tasks, FLIGHT’s reduction in run-time becomes even more pronounced (up to 64 $\times$) compared to other approaches, while maintaining the same level of backdoor resistance. Therefore, across tasks of any scale, FLIGHT enhances both efficiency and robustness, providing an effective robust secure aggregation solution for personalized federated learning.

7. Related Work

Poisoning Attack Defenses. To defend against poisoning attacks, existing robust defense methods commonly utilize detection algorithms to filter out poisoned updates. Methods such as Krum and M-Krum [5] leverage statistical differences, selecting the model updates with the smallest

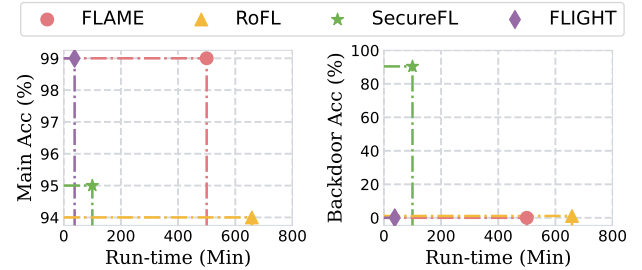


Figure 8: Robustness-Efficiency trade-off for various robust secure aggregation methods after 500 training rounds on training task MNIST. The x -axis represents the run-time and the y -axis represents the MA ($\uparrow$) and BA ($\downarrow$) after training.

Euclidean distances to the majority as the aggregation result. Trimmed Mean [56] removes the highest and lowest outliers in each parameter dimension across updates and computes the mean of the remaining values. While these approaches are effective against untargeted attacks that severely degrade the global model’s main task accuracy (MA), they struggle to defend against stealthy backdoor attacks that have minimal impact on MA. FLTrust [9] assumes access to a benign validation dataset and filters updates with large angular deviations from benign samples, but this assumption is difficult to achieve in practice and is ineffective against dynamic

poisoned updates with backdoor, especially under Non-IID conditions. Norm-based defenses [49] constrain the L_2 -norm of updates to mitigate untargeted and backdoor attacks, though overly strict norm bounds may inadvertently restrict benign updates, resulting in reduced MA. FLAME [41] employs density-based clustering (HDBSCAN [7]) for dynamic defenses, effectively addressing stealthy backdoor attacks in Non-IID scenarios, though this non-linear clustering method incurs high computational complexity.

Inference Attack Defenses. To mitigate inference attacks and prevent a compromised server from accessing model updates directly, many solutions employ cryptographic techniques for encrypting these updates, facilitating secure aggregation on the server side. Bonawitz *et al.* utilize the Diffie-Hellman algorithm [14], allowing clients to exchange paired additive masks to blind their model updates [6]. During aggregation, these masks cancel each other out, revealing the aggregated result. However, this extensive mask exchange incurs significant communication overhead. Aono *et al.* employ homomorphic encryption (HE) to encrypt model updates, enabling the server to perform additive aggregation directly on the homomorphic ciphertexts [2]. Nonetheless, HE causes ciphertext expansion, which limits communication efficiency. Additionally, secure multi-party computation (MPC) can be adapted for secure aggregation scenarios [12], [1]. Unlike the previous methods, MPC-based secure aggregation requires multiple servers as aggregators. Clients secret-share their model updates during the upload phase, and aggregators execute MPC protocols to compute the aggregated result. Since MPC supports complex non-linear operations, it is possible to design protocols that incorporate robust defense methods, achieving robust secure aggregation.

Robust Secure Aggregation (Hybrid Defenses). With the aim to preserve model update privacy and resist poisoning attacks from malicious clients, some works combine robust defense mechanism and cryptographic tools to propose the robust secure aggregation [36]. Earlier, MLGuard [27] combines naive Krum [5] with MPC to achieve robust secure aggregation. Recently, SecureFL [25] employ Secure Two-party Computation (2PC) protocols with linear complexity to implement the FLTrust [9] defense strategy, assuming that the server possesses a fixed validation dataset. However, like the defense methods on which these approaches are based, they fail to provide sufficient backdoor resistance in the Non-IID setting. RoFL [35] sets an L_2 -norm bound for model updates to exclude large norm poisoned updates. Furthermore, each client generates a ZKP proof regarding its L_2 -norm and uploads it to the server along with HE-encrypted model updates. The server verifies the ZKP proofs, and model updates satisfying the bound condition are aggregated under HE ciphertext. Similarly, ELSA [47] implements the L_2 -norm bound detection protocol using MPC. However, like RoFL, the overly strict norm boundaries restrict backdoor injections but lead to a significant drop in MA. In contrast, FLAME [41] designs 2PC protocols to implement HDBSCAN clustering [7] with the aim of effectively defending against both poisoning and inference

attacks simultaneously. Nevertheless, the complexity of the detection algorithm results in substantial communication overhead, thereby reducing aggregation efficiency. Compared to previous works, FLIGHT introduces (1) a two-stage personalized defense mechanism that effectively mitigates backdoor attacks in the Non-IID setting with lightweight complexity, and (2) a series of 2PC protocols based on this defense mechanism, which leverage their low complexity to achieve efficient and robust secure aggregation, providing both backdoor resistance and protection against inference attacks. This approach harmonizes robustness and efficiency, resulting in mutual enhancement. In addition, PBFL [38] and FLEDGE [11] combine blockchain with robust defense strategies, with the former using similarity detection techniques and the latter incorporating clustering techniques to filter poisoned updates. However, the decentralized structure is not friendly to edge client nodes with poor computational resources and the consensus algorithm requires significant network resources. In contrast, we focus on more realistic resource-constrained scenarios. Therefore, in this work, we do not compare with blockchain-based schemes.

8. Conclusion

In this paper, we first analyze the vulnerability of pFL to backdoor attacks and reveal that current robust secure aggregation methods fail to provide effective backdoor robustness for pFL while maintaining high aggregation efficiency. Consequently, we propose FLIGHT, a robust secure aggregation method for pFL. FLIGHT provides an efficient and privacy-preserving backdoor defense through its two-stage defense mechanism and communication-efficient 2PC protocols. Extensive experiments validate these improvements.

Limitations and Future Work. While FLIGHT provides strong security guarantees under the semi-honest adversary $\mathcal{A}^s$, under the assumption of a malicious adversary setting, the adversary $\mathcal{A}^s$ can arbitrarily deviate from the robust secure aggregation protocol, including altering the protocol's inputs and aggregation results, thereby fully controlling the output of the global model. To counter such advanced adversary, verification schemes such as Homomorphic Hash Function (HHF) [19] and Message Authentication Code (MAC) [3], [22] can be employed to verify the aggregation results, detect malicious activities, and abort the protocol in a timely manner, ensuring correctness during the aggregation process. However, applying these methods to robust detection algorithms requires significant precomputation and communication overhead with ciphertexts. Therefore, balancing the efficiency of aggregation with verification while defending against backdoor attacks is a direction for future expansion in FLIGHT.

Acknowledgements

We are grateful to the anonymous reviewers and shepherd for their valuable feedback and constructive comments. This work is supported by the Beijing Municipal Science & Technology Commission (Z231100005923047).

References

- [1] S. Addanki, K. Garbe, E. Jaffe, R. Ostrovsky, and A. Polychroniadou, "Prio+: Privacy preserving aggregate statistics via boolean shares," in *International Conference on Security and Cryptography for Networks*. Springer, 2022, pp. 516–539.
- [2] Y. Aono, T. Hayashi, L. Wang, S. Moriai *et al.*, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE transactions on information forensics and security*, vol. 13, no. 5, pp. 1333–1345, 2017.
- [3] T. Araki, A. Barak, J. Furukawa, T. Lichter, Y. Lindell, A. Nof, K. Ohara, A. Watzman, and O. Weinstein, "Optimized honest-majority mpc for malicious adversaries—breaking the 1 billion-gate per second barrier," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 843–862.
- [4] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *International conference on artificial intelligence and statistics*. PMLR, 2020, pp. 2938–2948.
- [5] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," *Advances in neural information processing systems*, vol. 30, 2017.
- [6] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.
- [7] R. J. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Pacific-Asia conference on knowledge discovery and data mining*. Springer, 2013, pp. 160–172.
- [8] R. Canetti, "Security and composition of multiparty cryptographic protocols," *Journal of CRYPTOLOGY*, vol. 13, pp. 143–202, 2000.
- [9] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "Fltrust: Byzantine-robust federated learning via trust bootstrapping," *arXiv preprint arXiv:2012.13995*, 2020.
- [10] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson *et al.*, "Extracting training data from large language models," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2633–2650.
- [11] J. Castillo, P. Rieger, H. Fereidooni, Q. Chen, and A. Sadeghi, "Fledge: Ledger-based federated learning resilient to inference and backdoor attacks," in *Proceedings of the 39th Annual Computer Security Applications Conference*, 2023, pp. 647–661.
- [12] H. Corrigan-Gibbs and D. Boneh, "Prio: Private, robust, and scalable computation of aggregate statistics," in *14th USENIX symposium on networked systems design and implementation (NSDI 17)*, 2017, pp. 259–282.
- [13] D. Demmler, T. Schneider, and M. Zohner, "Aby-a framework for efficient mixed-protocol secure two-party computation," in *NDSS*, 2015.
- [14] W. Diffie and M. E. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, 1976.
- [15] Y. Dong, X. Chen, K. Li, D. Wang, and S. Zeng, "Flod: Oblivious defender for private byzantine-robust federated learning with dishonest-majority," in *European Symposium on Research in Computer Security*. Springer, 2021, pp. 497–518.
- [16] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptology ePrint Archive*, 2012.
- [17] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to {Byzantine-Robust} federated learning," in *29th USENIX security symposium (USENIX Security 20)*, 2020, pp. 1605–1622.
- [18] U. Fiege, A. Fiat, and A. Shamir, "Zero knowledge proofs of identity," in *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, 1987, pp. 210–217.
- [19] D. M. Freeman, "Improved security for linearly homomorphic signatures: A generic framework," in *Public Key Cryptography–PKC 2012: 15th International Conference on Practice and Theory in Public Key Cryptography, Darmstadt, Germany, May 21–23, 2012. Proceedings 15*. Springer, 2012, pp. 697–714.
- [20] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients—how easy is it to break privacy in federated learning?" *Advances in neural information processing systems*, vol. 33, pp. 16 937–16 947, 2020.
- [21] O. Goldreich, S. Micali, and A. Wigderson, "How to play any mental game, or a completeness theorem for protocols with honest majority," in *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, 2019, pp. 307–328.
- [22] V. Goyal and Y. Song, "Malicious security comes free in honest-majority mpc," *Cryptology ePrint Archive*, 2020.
- [23] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, "Badnets: Evaluating backdooring attacks on deep neural networks," *IEEE Access*, vol. 7, pp. 47 230–47 244, 2019.
- [24] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: Private inference on transformers," *Advances in neural information processing systems*, vol. 35, pp. 15 718–15 731, 2022.
- [25] M. Hao, H. Li, G. Xu, H. Chen, and T. Zhang, "Efficient, private and robust federated learning," in *Proceedings of the 37th Annual Computer Security Applications Conference*, 2021, pp. 45–60.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [27] Y. Khazbak, T. Tan, and G. Cao, "Mlguard: Mitigating poisoning attacks in privacy preserving distributed collaborative learning," in *2020 29th international conference on computer communications and networks (ICCCN)*. IEEE, 2020, pp. 1–9.
- [28] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [29] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, pp. 541–551, 1989.
- [30] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [31] D. Li, H. Wang, R. Shao, H. Guo, E. Xing, and H. Zhang, "Mpc-former: Fast, performant and private transformer inference with mpc," in *The Eleventh International Conference on Learning Representations*, 2022.
- [32] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *International conference on machine learning*. PMLR, 2021, pp. 6357–6368.
- [33] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "Fedbn: Federated learning on non-iid features via local batch normalization," *arXiv preprint arXiv:2102.07623*, 2021.
- [34] P. P. Liang, T. Liu, L. Ziyin, N. B. Allen, R. P. Auerbach, D. Brent, R. Salakhutdinov, and L.-P. Morency, "Think locally, act globally: Federated learning with local and global representations," *arXiv preprint arXiv:2001.01523*, 2020.
- [35] H. Lycklama, L. Burkhalter, A. Viand, N. Küchler, and A. Hithnawi, "Rofl: Robustness of secure federated learning," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 453–476.
- [36] M. Mansouri, M. Önen, W. B. Jaballah, and M. Conti, "Sok: Secure aggregation based on cryptographic schemes for federated learning," *Proceedings on Privacy Enhancing Technologies*, 2023.
- [37] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

- [38] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo, and R. H. Deng, "Privacy-preserving byzantine-robust federated learning via blockchain systems," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2848–2861, 2022.
- [39] P. Mohassel and Y. Zhang, "Secureml: A system for scalable privacy-preserving machine learning," in *2017 IEEE symposium on security and privacy (SP)*. IEEE, 2017, pp. 19–38.
- [40] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "Diot: A federated self-learning anomaly detection system for iot," in *2019 IEEE 39th International conference on distributed computing systems (ICDCS)*. IEEE, 2019, pp. 756–767.
- [41] T. D. Nguyen, P. Rieger, R. De Viti, H. Chen, B. B. Brandenburg, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen *et al.*, "{FLAME}: Taming backdoors in federated learning," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 1415–1432.
- [42] M. S. Ozdayi, M. Kantarcioglu, and Y. R. Gel, "Defending against backdoors in federated learning with robust learning rate," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 10, 2021, pp. 9268–9276.
- [43] K. Pillutla, K. Malik, A.-R. Mohamed, M. Rabbat, M. Sanjabi, and L. Xiao, "Federated learning with partial model personalization," in *International Conference on Machine Learning*. PMLR, 2022, pp. 17716–17758.
- [44] S. R. Pokhrel and J. Choi, "Federated learning with blockchain for autonomous vehicles: Analysis and design challenges," *IEEE Transactions on Communications*, vol. 68, no. 8, pp. 4734–4746, 2020.
- [45] Z. Qin, L. Yao, D. Chen, Y. Li, B. Ding, and M. Cheng, "Revisiting personalized federated learning: Robustness against backdoor attacks," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 4743–4755.
- [46] D. Rathee, M. Rathee, R. K. K. Goli, D. Gupta, R. Sharma, N. Chandran, and A. Rastogi, "Sirnn: A math library for secure rnn inference," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1003–1020.
- [47] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "Elsa: Secure aggregation for federated learning with malicious actors," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1961–1979.
- [48] N. Rieke, J. Hancox, W. Li, F. Milletari, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein *et al.*, "The future of digital health with federated learning," *NPJ digital medicine*, vol. 3, no. 1, pp. 1–7, 2020.
- [49] Z. Sun, P. Kairouz, A. T. Suresh, and H. B. McMahan, "Can you really backdoor federated learning?" *arXiv preprint arXiv:1911.07963*, 2019.
- [50] C. T. Dinh, N. Tran, and J. Nguyen, "Personalized federated learning with moreau envelopes," *Advances in Neural Information Processing Systems*, vol. 33, pp. 21 394–21 405, 2020.
- [51] J.-L. Watson, S. Wagh, and R. A. Popa, "Piranha: A {GPU} platform for secure computation," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 827–844.
- [52] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [53] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "Dbas: Distributed backdoor attacks against federated learning," in *International conference on learning representations*, 2019.
- [54] Y. Xie, Z. Wang, D. Gao, D. Chen, L. Yao, W. Kuang, Y. Li, B. Ding, and J. Zhou, "Federatedscope: A flexible federated learning platform for heterogeneity," *Proceedings of the VLDB Endowment*, vol. 16, no. 5, pp. 1059–1072, 2023.
- [55] H. Yan, W. Zhang, Q. Chen, X. Li, W. Sun, H. Li, and X. Lin, "Recess vaccine for federated learning: Proactive defense against model poisoning attacks," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [56] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *International conference on machine learning*. Pmlr, 2018, pp. 5650–5659.

Appendix A. Notations

The Table 3 displays the notations used in this paper.

TABLE 3: Notation table.

C_i	Client $i \in [1, n]$
S_j	Server $j \in [1, 2]$
x	Lowercase letter denotes scalar
$\mathbf{x}$	Lowercase bold letter denotes vector
$[\cdot]$	Additional secret sharing
$\mathbb{Z}_{2^\ell}$	Discrete ring modulo 2^ℓ with ℓ -bits
n	Number of total clients
m	Number of malicious clients
η	Learning rate
θ_p	Threshold of validation model upgrading
θ_{exp}	Threshold of negExp function

Appendix B.

Details of Protocol Π_{NT} and $\Pi_{Softmax}$

Protocol 1 and 2 show the details of Π_{NT} (Norm Temperature) and $\Pi_{Softmax}$ (Softmax):

Protocol 1: Norm temperature Π_{NT}

Input: $S_j, j \in [1, 2]$ holds $\llbracket cos_i \rrbracket_j, \llbracket l_i \rrbracket_j$ for $i \in [1, n]$.

Output: S_j gets $\llbracket T_i \rrbracket_j$, where $T_i = (\text{Mean}(l_i)/l_i)^{\text{sgn}(cos_i)}$.

```

1  $\llbracket m \rrbracket = \Pi_{\text{Mean}}(\llbracket l_{i \in [1, n]} \rrbracket)$ .
2  $\llbracket 1/m \rrbracket = \Pi_{\text{Recip}}(\llbracket m \rrbracket)$ .
3 for client  $i \in [1, n]$  in parallel do
4    $\llbracket 1/l_i \rrbracket = \Pi_{\text{Recip}}(\llbracket l_i \rrbracket)$ .
5    $\llbracket b_i \rrbracket^B = \Pi_{\text{Cmp}}(\llbracket cos_i \rrbracket)$ .
6    $\llbracket l_i/m \rrbracket = \Pi_{\text{Mul}}(\llbracket l_i \rrbracket, \llbracket 1/m \rrbracket)$ ,
    $\llbracket m/l_i \rrbracket = \Pi_{\text{Mul}}(\llbracket m \rrbracket, \llbracket 1/l_i \rrbracket)$ .
7    $\llbracket T_i \rrbracket =$ 
    $\Pi_{\text{MulAB}}((\llbracket m/l_i \rrbracket - \llbracket l_i/m \rrbracket), \llbracket b_i \rrbracket^B) + \llbracket l_i/m \rrbracket$ .
8 end
9 return  $\llbracket T_i \rrbracket$ .
```

In the protocol Π_{NT} , given the cosine similarity vector $\llbracket cos_{i \in [1, n]} \rrbracket$ and the norm vector $\llbracket l_{i \in [1, n]} \rrbracket$ as inputs, we first use protocol $\Pi_{\text{Mean}}(\llbracket l_{i \in [1, n]} \rrbracket)$ to compute the mean $\llbracket m \rrbracket$ of the norm vector, followed by using protocol $\Pi_{\text{Recip}}(\llbracket m \rrbracket)$ to compute its reciprocal $\llbracket 1/m \rrbracket$. Next, we apply protocol $\Pi_{\text{Recip}}(\llbracket l_{i \in [1, n]} \rrbracket)$ to calculate the reciprocal of each value in the norm vector and use protocol $\Pi_{\text{Cmp}}(\llbracket cos_i \rrbracket)$ to determine whether $cos_i \geq 0$. If it is, returns $b_i = 1$; otherwise, returns

$b_i = 0$. Then, we use multiplication protocol Π_{Mul} to compute values $\llbracket l_i/m \rrbracket$ and $\llbracket m/l_i \rrbracket$, and finally, the multiplexer constructed by protocol $\Pi_{\text{MulAB}}((\llbracket m/l_i \rrbracket - \llbracket l_i/m \rrbracket), \llbracket b_i \rrbracket^B) + \llbracket l_i/m \rrbracket$ determines the output norm temperature $\llbracket T_i \rrbracket$: if $b_i = 1$, it returns $\llbracket m/l_i \rrbracket$; if $b_i = 0$, it returns $\llbracket l_i/m \rrbracket$.

Protocol 2: Softmax Π_{Softmax}

Input: $S_j, j \in [1, 2]$ holds $\llbracket ps_i \rrbracket_j$ for $i \in [1, n]$, where $ps_i = \cos i \cdot T_i$.

Output: S_j gets $\llbracket \text{Softmax}(\gamma_i) \rrbracket_j$.

```

1  $\llbracket ps_{\max} \rrbracket = \Pi_{\text{Max}}(\llbracket ps_i \rrbracket_{i \in [1, n]})$ .
2 for client  $i \in [1, n]$  in parallel do
3    $\llbracket b_i \rrbracket^B = \Pi_{\text{Cmp}}(\llbracket ps \rrbracket - \theta_{\text{exp}})$ .
4    $\llbracket z_i \rrbracket = 1 + \llbracket ps_i - ps_{\max} \rrbracket$ .
5    $\llbracket z_i \rrbracket = \Pi_{\text{MulAB}}(\llbracket z_i \rrbracket, \llbracket b_i \rrbracket^B)$ .
6    $\llbracket z_i^2 \rrbracket = \Pi_{\text{Mul}}(\llbracket z_i \rrbracket, \llbracket z_i \rrbracket)$ .
7    $\llbracket z_{\text{sum}}^2 \rrbracket = \sum_{i=1}^n (\llbracket z_i^2 \rrbracket)$ .
8    $\llbracket 1/z_{\text{sum}}^2 \rrbracket = \Pi_{\text{Recip}}(\llbracket z_{\text{sum}}^2 \rrbracket)$ .
9    $\llbracket z_i^2/z_{\text{sum}}^2 \rrbracket = \Pi_{\text{Mul}}(\llbracket z_i^2 \rrbracket, \llbracket 1/z_{\text{sum}}^2 \rrbracket)$ .
10   $\llbracket \gamma_i \rrbracket = \llbracket z_i^2/z_{\text{sum}}^2 \rrbracket$ .
11 end
12 return  $\llbracket \gamma_i \rrbracket$ .
```

In the protocol Π_{Softmax} , given the personalized score vector $\llbracket ps_{i \in [1, n]} \rrbracket$ as input, we first use protocol $\Pi_{\text{Max}}(\llbracket ps_{i \in [1, n]} \rrbracket)$ to compute the maximum value $\llbracket ps_{\max} \rrbracket$ in the vector. Next, for each value $\llbracket ps_i \rrbracket$ in the vector, we compare it with the threshold θ_{exp} : if it exceeds the threshold, we set $b_i = 1$; otherwise $b_i = 0$. We then calculate $1 + \llbracket ps_i - ps_{\max} \rrbracket$, multiply it by $\llbracket b_i \rrbracket^B$, and use this as the input $\llbracket z_i \rrbracket$ for the softmax computation. For each input $\llbracket z_i \rrbracket$, we approximate its softmax output using the 2Quad method: protocol $\Pi_{\text{Mul}}(\llbracket z_i \rrbracket, \llbracket z_i \rrbracket)$ is used to compute the square $\llbracket z_i^2 \rrbracket$ of each input, followed by summing the squared results and calculating its reciprocal $\llbracket 1/z_{\text{sum}}^2 \rrbracket$. Finally, the multiplication protocol $\Pi_{\text{Mul}}(\llbracket z_i^2 \rrbracket, \llbracket 1/z_{\text{sum}}^2 \rrbracket)$ is applied to obtain the final output $\llbracket \gamma_i \rrbracket$.

Appendix C.

Proof of Security and Correctness

C.1. Proof of Theorem 1

Proof. We analyze FLIGHT securely realize the protocols Π_{Cos} , Π_{NT} , and Π_{Softmax} against the semi-honest $\mathcal{A}^s$, ensuring $\mathcal{A}^s$ cannot infer anything beyond $\mathbf{w}_g$. Given the ideal protocols Π_{Mul} , Π_{MulAB} , Π_{Mean} , Π_{Recip} , Π_{Cmp} , and Π_{Max} defined in [46], we now outline the security proof of Π_{Cos} , Π_{NT} , and Π_{Softmax} in the stand-alone model under sequential composition [8], and show that the view of S_j corrupted by adversary $\mathcal{A}^s$ in the real-world is computational indistinguishable of to the ideal-world simulation of adversary (simulator) Sim :

Security of Π_{Cos} : Since Π_{Cos} relies solely on the underlying Π_{Mul} , we faithfully executes Π_{Mul} to achieve Π_{Cos} and the security of Π_{Cos} also stems from this underlying

protocol. Given that the protocol Π_{Mul} is secure, the view of S_j in the real world is computationally indistinguishable from that in the ideal world.

Security of Π_{NT} : The security of Π_{NT} relies on the ideal $(\Pi_{\text{Mean}}, \Pi_{\text{Recip}}, \Pi_{\text{Cmp}}, \Pi_{\text{Mul}}, \Pi_{\text{MulAB}})$ -hybrid model. In the real-world, we black-box access the hybrid model. In the ideal-world, the output of Sim for S_j is randomly chosen from $\mathbb{Z}_{2^\ell}$. Thus, S_j 's view in real-world is computationally indistinguishable from that in ideal-world.

Security of Π_{Softmax} : In the real-world, we black-box access ideal $\Pi_{\text{Max}}, \Pi_{\text{Recip}}, \Pi_{\text{Cmp}}, \Pi_{\text{Mul}}, \Pi_{\text{MulAB}}$ to get $\llbracket \gamma_i \rrbracket$. While, Sim samples a random number $r \in_R \mathbb{Z}_{2^\ell}$ in the ideal-world, and sends it to S_j as the sharing of $\llbracket \gamma_i \rrbracket$. As the $(\Pi_{\text{Max}}, \Pi_{\text{Recip}}, \Pi_{\text{Cmp}}, \Pi_{\text{Mul}}, \Pi_{\text{MulAB}})$ -hybrid model is secure, the view of S_j in real-world is also computationally indistinguishable from that in ideal-world.

Therefore, we guarantee that the adversary $\mathcal{A}^s$ infers nothing beyond the aggregation result $\mathbf{w}_g$. $\square$

C.2. Proof of Theorem 2

Proof. To demonstrate that the ciphertext operations yield the same backdoor-resistant effectiveness as plaintext operations, we need to verify the correctness of FLIGHT:

The correctness of FLIGHT relies on the $\llbracket \cdot \rrbracket$ -sharing semantics and the cryptographic protocols $\Pi_{\text{Cos}}, \Pi_{\text{NT}}, \Pi_{\text{Softmax}}$, which are based on ideal protocols defined in SIRNN [46]. The correctness of $\llbracket \cdot \rrbracket$ -sharing is evident [13], [39], and similarly, the ideal protocols $\Pi_{\text{Mul}}, \Pi_{\text{MulAB}}, \Pi_{\text{Mean}}, \Pi_{\text{Recip}}, \Pi_{\text{Cmp}}$, and Π_{Max} are also correct. Furthermore, 1) Π_{Cos} correctly computes the cosine similarity based on Π_{Mul} . 2) $(\Pi_{\text{Mean}}, \Pi_{\text{Recip}}, \Pi_{\text{Cmp}}, \Pi_{\text{Mul}}, \Pi_{\text{MulAB}})$ -hybrid protocol ensures S_j gets the result $\llbracket T_i \rrbracket_j, T_i = (\text{Mean}(l_{i \in [1, n]}))/l_i^{\text{sgn}(\cos i)}$, since the $\llbracket \cdot \rrbracket$ -sharing is correct, the protocol Π_{NT} can correctly compute the norm temperature of l_i . 3) Π_{Softmax} correctly computes the softmax based on $(\Pi_{\text{Max}}, \Pi_{\text{Recip}}, \Pi_{\text{Cmp}}, \Pi_{\text{Mul}}, \Pi_{\text{MulAB}})$ -hybrid protocol. $\square$

Appendix D.

Supplementary Materials for Aggregation Cost

D.1. Details of FLIGHT's Aggregation Cost

Table 4 presents the total aggregation cost of FLIGHT under various inputs, including a wide range of models and client numbers, consistent with the total costs depicted in Figure 6. For different neural network models, we ensure that 10% of the 100 clients are selected for aggregation in each round. When varying the number of clients participating in the aggregation, we consistently use the MNIST training task for evaluation.

Table 5 shows the relationship between the execution efficiency of various protocols in FLIGHT-Stage2 and the number of clients n . We observe that the communication and run-time of each protocol increase linearly with n . The main communication overhead arises from the weight uploads and

TABLE 4: Communication cost (GB) and Run-time (Min) of total aggregation cost of FLIGHT on varying inputs. For different network models, the number of clients selected to participate in aggregation is fixed as 10. For different number of clients, the training task is fixed as MNIST (62k).

Efficiency Metrics	Network Models			Number of Clients		
	MNIST	CIFAR-10S	CIFAR-10L	25 clients	50 clients	100 clients
Comm	42.58	142.83	679.02	58.31	159.79	452.67
Run-time	36.50	63.79	158.15	49.82	73.52	139.96

TABLE 5: Communication cost (GB) and Run-time (Min) of each protocol of FLIGHT-Stage2. The model parameter is fixed as 50k and each row represents the number of clients selected to participate in the aggregation.

Client (n)	C-S Phase (WAN)				S-S Phase (LAN)							
	Upload		Download		Cosine Similarity		Norm Temperature		Softmax		Aggregation	
	Comm	Run-time	Comm	Run-time	Comm	Run-time	Comm	Run-time	Comm	Run-time	Comm	Run-time
50	18.37	7.51	9.18	4.62	9.40	1.75	0.42	21.45	0.02	16.50	9.41	1.06
100	36.74	14.27	18.37	8.33	18.62	2.29	0.84	31.35	0.03	19.80	18.65	1.67
150	55.59	21.53	27.55	10.80	27.84	2.74	1.26	42.07	0.04	24.75	27.87	2.49

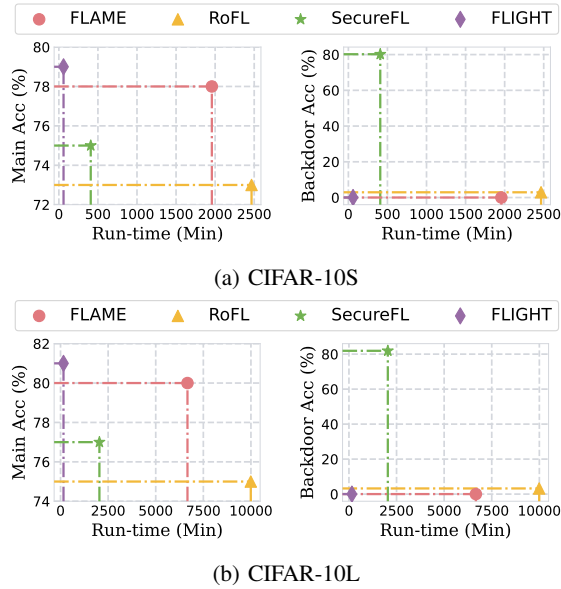


Figure 9: Robustness-Efficiency trade-off for various robust secure aggregation methods after 500 training rounds on training task CIFAR-10S and CIFAR-10L. The x -axis represents the run-time and the y -axis represents the MA ($\uparrow$) and BA ($\downarrow$) after training.

downloads in the C-S phase, and from the cosine similarity and aggregation in the S-S phase. The primary run-time overhead comes from the Norm Temperature (NT) and Softmax operations in the S-S phase. This is because NT and Softmax require numerous MPC non-linear operations, resulting in a significant number of low-communication interactions. The increase in interaction rounds leads to longer

run-time. In contrast, the cosine similarity and aggregation operations are based on matrix multiplication. FLIGHT leverages GPU-accelerated ciphertext matrix operations to speed up these operations, thereby reducing the overall run-time.

D.2. Additional Robustness-Efficiency Trade-off Experiment

Figure 9 shows the additional Robustness-Efficiency trade-off for different robust secure aggregation methods in defending against *Badnet Attack* on CIFAR-10S and CIFAR-10L training tasks. For these tasks, we find that FLIGHT also achieves backdoor defense capability comparable to FLAME without sacrificing MA, while significantly reducing run-time by approximately $7\text{-}64\times$. The trade-off performance of the other approaches is consistent with that observed in the MNIST task, struggling to balance efficiency and robustness. Therefore, as the scale of the task increases, FLIGHT increasingly balances efficiency and robustness, leading to superior outcomes in both.